Blastline

Product Definition for HackHydra Track 2A — Supply-Chain Blast Radius

Blastline connects arbitrary JavaScript/TypeScript repositories into a provider-independent dependency graph, preserves their dependency state over time, and lets defenders investigate a compromised package across both their own software estate and the wider npm ecosystem.

1. The Useful Problem We Solve

The existing landscape is good at repository-level vulnerability detection, but weak at incident-time, cross-repository context. A developer can scan one lockfile and learn which dependencies are vulnerable, whether they are direct or transitive, and what upgrade may fix them. The harder question begins after a new supply-chain incident is announced.

The defender needs to answer, quickly:

- Which tracked repositories actually resolved the compromised package version?
- Which dependency path brought that package into each repository?
- Which repository snapshots were affected during the incident window, which were clean, and where is the state unknown?
- What is the complete reverse-dependency blast radius in the npm ecosystem?
- Which other packages are connected through the same maintainer or publisher identity?
- Are there suspicious package names that look like likely typosquats?
- If many repositories are affected, what small set of parent upgrades removes the most shared risk?

This is fundamentally a graph problem. The answer depends on exact version-to-version dependency relationships, lockfile resolutions, repository snapshots, maintainer relationships and time windows. Semantic similarity alone cannot compute transitive reverse closure or prove that a particular lockfile resolved a bad version.

2. Where Blastline Fits in the Existing Tooling Landscape

Tool / Layer	What it is good at	Blastline's boundary
CVE Lite CLI	Deep dependency and vulnerability analysis inside one JS/TS repository.	Use its intelligence; do not rebuild its scanner.
Git / CI	Tells us when repository state changes and gives commit identity.	Use it to trigger and identify immutable security snapshots.
gittuf principles	Provider-independent repository trust, network/controller patterns and append-only security thinking.	Borrow the architectural principles, not gittuf's policy engine.
HydraDB	Graph-native storage and	Use as the reasoning layer for

Tool / Layer	What it is good at	Blastline's boundary
	traversal across versioned relationships.	blast radius and incident investigation.
Blastline	Cross-repo + ecosystem incident context.	Connect all of the above into one incident-response product.

3. How Blastline Solves the Problem

Blastline uses two connected graphs:

```
PUBLIC NPM ECOSYSTEM GRAPH

PackageVersion
   ↓ DEPENDS_ON
PackageVersion
   ↓ MAINTAINED_BY
Maintainer

        connected to

WORKSPACE GRAPH

Workspace
   ↓ CONTAINS
Repository
   ↓ HAS_SNAPSHOT
SecuritySnapshot
   ↓ RESOLVED
PackageVersion
```

The public ecosystem graph answers potential impact: if package X is compromised, what package versions depend on it directly or transitively?

The workspace graph answers observed impact: which of the repositories we track actually resolved that exact package version in an immutable lockfile snapshot?

The connection between the two is the exact PackageVersion node. That lets a defender move from global blast radius to their own confirmed exposure without depending on a GitHub Organization.

4. Provider-Independent Workspace Model

Blastline does not use a GitHub Organization as the root security boundary. Instead, it creates a logical Workspace with a stable UUID. Any repository can be linked to that workspace regardless of where it is hosted.

```
Workspace: ws_019c72...

├── cadence            GitHub / Figmenta
├── payments-api       GitHub / another org
├── client-dashboard   GitLab
└── internal-worker    local/private
```

- Workspace UUID is stable and provider-independent.
- Each repository gets its own stable repository UUID.
- Remote URL is metadata, not identity.
- Moving a repo from GitHub to GitLab does not destroy its Blastline history.
- The same workspace can contain company repos, client repos and local/private repos.

5. Features We Reuse From CVE Lite CLI

CVE Lite is the repository-level intelligence engine. Blastline should consume its structured output rather than duplicate these capabilities.

CVE Lite capability	Why we reuse it	How Blastline extends it
Lockfile parsing	Already handles JS/TS dependency state.	Store the result as a workspace snapshot.
npm / pnpm / Yarn / Bun support	Avoid rebuilding package-manager parsers.	Normalize all repos into the same graph model.
OSV vulnerability matching	Established vulnerability source.	Aggregate findings across many repos.
Direct vs transitive classification	Explains local dependency position.	Use it in cross-repo blast-radius views.
Dependency paths	Shows how a package entered a repo.	Compare common dependency roots across repos.
Fix / parent-aware remediation	Already reasons about useful upgrades.	Group repeated fixes into workspace-wide remediation campaigns.
Maintenance-risk DM001	Finds direct packages blocking fixes for vulnerable transitives.	Find the same maintenance bottleneck across many repos.
JSON / CycloneDX output	Gives machine-readable scan data.	Use as ingestion boundary into Blastline.
Ratcheting concept	Tracks newly introduced findings inside a repo.	Extend to workspace-wide security drift over time.
Override hygiene	Adds dependency-health signals.	Aggregate shared hygiene problems across repositories.

6. Principles We Borrow From gittuf

We borrow architecture principles, not gittuf's repository policy implementation.

gittuf principle	Blastline adaptation	Why it matters
Provider-independent repository network	Workspace can contain repos from any provider.	We do not depend on GitHub Organization boundaries.
Controller / network thinking	Workspace is the logical root that groups repositories.	Cross-repo security context remains explicit.

gittuf principle	Blastline adaptation	Why it matters
Stable security identity	Workspace UUID + repository UUID.	History survives repository moves and remote URL changes.
Two-way membership idea	Workspace tracks repo; repo stores its workspace/repo IDs.	Makes local linking and verification predictable.
Append-only activity mindset	Every scan becomes a new immutable SecuritySnapshot.	Historical lockfile state remains available for incident windows.
Automatic recording principle	Capture snapshots from Git push/CI instead of relying only on manual scans.	Security history stays fresh with normal developer workflow.

7. Git Push / CI Role

Git is not our security scanner. It gives Blastline a trustworthy change event and commit identity.

```
Developer push / merge
        ↓
CI runs blastline scan
        ↓
CVE Lite analyzes repository
        ↓
Blastline records:
- repository UUID
- commit SHA
- scan timestamp
- lockfile hash
- exact resolved packages
- vulnerability / maintenance findings
        ↓
Immutable SecuritySnapshot
        ↓
HydraDB
```

Why snapshots matter: if a package is later reported malicious for a known time window, Blastline can inspect historical repository state rather than only today's lockfile.

8. Features Relevant to This Hackathon Build

The build should stay focused on the Track 2A problem. The following features are directly relevant and should define the MVP.

Feature	Purpose	Priority
Workspace UUID	Group arbitrary repos without relying on GitHub Organization.	P0
Stable repository UUID	Keep repo identity independent of remote provider.	P0
blastline link	Attach a local repo to a workspace.	P0

Feature	Purpose	Priority
blastline scan	Run CVE Lite + collect Git identity + submit normalized data.	P0
Immutable SecuritySnapshot	Preserve exact dependency state over time.	P0
Cross-repo package lookup	Find every tracked repo resolving package@version.	P0
Manual malicious-package incident	Create an incident for an affected version + time window.	P0
Affected / Clean / Unknown	Report evidence without treating missing data as safe.	P0
Dependency path explanation	Show how the compromised package enters each repo.	P0
Basic npm ecosystem graph	Represent package-version dependencies in HydraDB.	P0
Reverse dependency closure	Compute full ecosystem blast radius from a compromised version.	P0
Common exposure root	Find the parent dependency shared by many affected repos.	P1
Remediation campaigns	Aggregate CVE Lite fixes into highest-leverage cross-repo actions.	P1
Maintenance bottleneck aggregation	Find shared direct dependencies blocking security fixes.	P1
Workspace security drift	Show new/fixed exposure across snapshots.	P1
Shared maintainer traversal	Find other packages connected to a compromised maintainer.	P1
Typosquat candidates	Flag suspicious names near popular packages.	P2
Push/CI automatic snapshots	Continuously refresh security state during normal development.	P2

9. Explicit Non-Goals for This Build

These may be useful later, but they dilute the Track 2A story and should not be part of the first build:

- Deployment/runtime provenance.
- Kubernetes, AWS or Vercel integrations.
- Function-level runtime reachability.
- Malware sandboxing or behavioral malware detection.
- SAST, secret scanning or container scanning.
- Full enterprise RBAC.

- Jira/Slack workflow integrations.
- AI chat as a core feature.
- Automatic pull-request generation.
- PyPI support before the npm graph is working well.

10. Core User Flow

```
1. Create workspace
   blastline workspace create engineering

2. Link arbitrary repository
   cd repo
   blastline link <workspace-id>

3. Scan repository
   blastline scan

4. Repeat for other repositories from any provider

5. CI / future pushes create additional immutable snapshots

6. A package incident is reported:
   package foo@2.1
   malicious 09:00 → 09:06

7. Blastline queries HydraDB:
   - ecosystem reverse dependency closure
   - tracked snapshots resolving foo@2.1
   - dependency path per repository
   - common parent / maintenance bottlenecks
   - related maintainer packages

8. Defender receives:
   - ecosystem blast radius
   - confirmed affected repos
   - confirmed clean repos
   - unknown repos
   - shared remediation opportunities
```

11. Where This Becomes Important

- Teams maintaining many JavaScript/TypeScript repositories spread across different providers or clients.
- Security incidents where a package version is suddenly reported malicious and defenders need an immediate blast-radius answer.
- Organizations where the same vulnerable or deprecated parent dependency appears across many services.
- Incident response where historical lockfile state matters more than today's repository state.
- Engineering teams that want provider-independent, self-hostable dependency intelligence instead of tying security state to one Git forge.
- Security teams that need explicit evidence: affected, clean or unknown - not a flat vulnerability count.

12. Final Product Boundary

CVE Lite	Understands one repository: dependencies, vulnerabilities, paths, maintenance risk and remediation.
gittuf principles	Teach us how security context can span arbitrary repositories with stable identities and historical state.
Git / CI	Tells us when repository state changes and when to capture another snapshot.
Blastline + HydraDB	Connects ecosystem and workspace relationships to answer supply-chain incident blast radius and coordinated remediation.

Blastline does not replace CVE Lite. It turns repository-level dependency intelligence into a living, provider-independent security graph for incident response.

Reference material used for product principles: OWASP CVE Lite CLI documentation, gittuf multi-repository/design documentation, Git hooks/CI workflow concepts, and the HackHydra Track 2A supply-chain blast-radius problem statement.